

IST602 — Web Technologies and Standards

Second Semester Examination 2024/2025 — Model Answers

Institution: University of Buea, Faculty of Science, Department of Computer Science

Course: IST602 – Web Technologies and Standards

Instructor: Denis Nkweteyim

Date: 26/06/2025 | **Time:** 08:00–11:00 (3 hours)

Venue: G-Block | **Credit value:** 6

Instructions on paper: Attempt all three questions.

Question 1 — XML (25 marks)

(a) (8 marks)

(i) Three syntax rules an XML document must conform to (any three)

1. **Exactly one root element** — all other elements must be nested inside a single root.
2. **Every element must have a matching closing tag** (or use correct self-closing syntax for empty elements).
3. **Elements must be properly nested** — tags must not overlap incorrectly (e.g. `<a><b></a></b>` is invalid).
4. **XML tags are case-sensitive** — `<Note>` and `<note>` are different.
5. **Attribute values must be quoted** — e.g. `Language="English"`.

(Give any three with brief explanation.)

(ii) What is a well-formed XML document?

A **well-formed** XML document is one that **follows all the basic XML syntax rules** defined by the W3C XML Recommendation (single root, proper nesting, matching tags, quoted attributes, etc.). A parser can read and process a well-formed document structurally.

(iii) What is a valid XML document?

A **valid** XML document is one that is:

1. **Well-formed**, and
2. **Conforms to a document type definition** — either a **DTD** or an **XML Schema (XSD)** — that specifies allowed elements, attributes, order, and content types.

Validation checks that the document matches the declared grammar/contract, not just syntax.

(iv) Are Figure 1 and Figure 2 well-formed?

Figure 1 — YES, well-formed

```
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me today</body>
</note>
```

- Has **one root element** (`note`).
- All elements are properly opened and closed and nested inside `note`.

Figure 2 — NO, not well-formed

```
<?xml version="1.0"?>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me today</body>
```

- **Reason:** It has **multiple root-level elements** (`to`, `from`, `heading`, `body`). XML allows only **one root element** besides the optional XML declaration/prolog. These sibling elements at the top level violate the well-formedness rule.

(b) (13 marks)

(i) What are DTDs and Schemas used for?

DTDs and **XML Schemas (XSD)** are used to **define the structure and constraints** of an XML vocabulary (which elements and attributes are allowed, in what order, how many times, and what data they may contain). Applications use them to **validate** that an XML instance document conforms to the expected format before processing.

(ii) Three problems with DTDs that XML Schemas address

1. **Weak data typing** — DTDs mainly use `#PCDATA/CDATA`; XSD supports rich types (`integer`, `date`, `decimal`, `boolean`, etc.) and facets (min/max, patterns, enumerations).
2. **Non-XML syntax** — DTDs use a separate grammar; **XSD is XML-based**, so the same tools can parse and process schemas.
3. **Limited namespace support** — DTD namespace handling is awkward; **XSD has built-in namespace support** for combining vocabularies.
4. (Alternative) **No support for default/fixed values and constraints** as expressively as XSD; XSD supports `minOccurs`/`maxOccurs`, extensions, restrictions, and clearer attribute rules.

(Any three distinct points.)

(iii) Explain each line of the DTD

```
<!DOCTYPE bibliography [
  <!ELEMENT bibliography (book+)>
  <!ELEMENT book (title, author*, publisher?, year?, section*)>
  <!ATTLIST book ISBN CDATA #REQUIRED>
  <!ATTLIST book price CDATA #IMPLIED>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT publisher (#PCDATA)>
  <!ELEMENT year (#PCDATA)>
  <!ELEMENT i (#PCDATA)>
  <!ELEMENT content (#PCDATA|i)*>
  <!ELEMENT section (title, content?, section*)>
]>
```

Line	Meaning
<!DOCTYPE bibliography [	Starts an internal DTD ; root element of valid documents is bibliography .
<!ELEMENT bibliography (book+)>	bibliography must contain one or more book elements, in that order.
<!ELEMENT book (title, author*, publisher?, year?, section*)>	Each book contains: one title , zero or more author , optional publisher (?), optional year (?), zero or more section .
<!ATTLIST book ISBN CDATA #REQUIRED>	book has attribute ISBN (character data); must be present.
<!ATTLIST book price CDATA #IMPLIED>	book may have optional attribute price ; if omitted, parser may ignore or assign default.
<!ELEMENT title (#PCDATA)>	title contains parsed character data (text).
<!ELEMENT author (#PCDATA)>	author contains text.
<!ELEMENT publisher (#PCDATA)>	publisher contains text.
<!ELEMENT year (#PCDATA)>	year contains text.
<!ELEMENT i (#PCDATA)>	i (italic markup) contains text.
<!ELEMENT content (#PCDATA i)*>	content contains zero or more mix of text (#PCDATA) and i elements.
<!ELEMENT section (title, content?, section*)>	section has one title , optional content , and zero or more nested section (recursive structure).
>	End of internal DTD.

(c) (4 marks) — Bookstore table as XML

Requirements: Title has attribute **Language**; other fields have no attributes. Book 3 has three authors.

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book>
    <Title Language="English">Everyday Italian</Title>
    <Author>Giada De Laurentiis</Author>
    <Year>2005</Year>
    <Price>30.00</Price>
    <Category>Cooking</Category>
  </book>
  <book>
    <Title Language="English">Harry Potter</Title>
    <Author>J.K. Rowling</Author>
    <Year>2005</Year>
    <Price>29.99</Price>
    <Category>Children</Category>
  </book>
  <book>
    <Title Language="English">XQuery Kick Start</Title>
    <Author>James McGovern</Author>
    <Author>Per Bothner</Author>
    <Author>Kurt Cagle</Author>
    <Year>2003</Year>
    <Price>49.99</Price>
    <Category>Web</Category>
  </book>
</bookstore>
```

(Element names may vary if consistent; key points: one root, Title + Language attribute, multiple Author elements for book 3.)

Question 2 — TypeScript (25 marks)

(a) (2 + 2 marks)

What does a transpiler do?

A **transpiler** (source-to-source compiler) reads code written in one language and produces **equivalent code in another language** at a similar abstraction level — without executing the program.

What does a TypeScript file compile to?

A TypeScript (**.ts**) file compiles to **plain JavaScript** (**.js**), which browsers and Node.js can run.

(b) (2 × 4 = 8 marks) — Are the functions valid?

(i) `function myFunc(x:number, y:string): string { }`

Yes — valid.

- Parameters: `x: number`, `y: string`
 - Return type: `string`
 - (Body is empty; at runtime would return *undefined* unless strict checks catch unreachable paths — but the **signature** is valid.)
-

(ii) `function myFunc(x:number, y:string) { }`

Yes — valid.

- Parameters: `x: number`, `y: string`
 - Return type: **inferred** as `void` (no return statement)
-

(iii) `function myFunc(pt: { x: number; y: number }) { }`

Yes — valid.

- Parameter: `pt` — inline object type with `x: number`, `y: number`
 - Return type: inferred `void`
-

(iv) `function myFunc(x:number, y): number { }`

No — not valid (under normal TypeScript strict settings).

- **Problem:** Parameter `y` has **no type annotation**. In TypeScript, parameters require explicit types unless a default value or rest pattern applies; the compiler error is typically *"Parameter 'y' implicitly has an 'any' type"* (or requires a type).
-

(c) (3 × 2 = 6 marks) — Are the calls valid?

(i)

```
function intro(name:String, id:number) {  
    console.log("Hello, ", name, " has ID ", id);  
}  
intro("Brendan")
```

No — invalid call.

- **Problem:** `intro` requires **two arguments** (`name` and `id`), but only `"Brendan"` is supplied. Missing argument `id`.
-

(ii)

```
function showNames(name1:string, ...names:string[]) { ... }  
showNames("Ruth", "Bob", "Guy", "Anita");  
showNames("Ruth", "Bob", "Guy", "Anita", "Mary");
```

Yes — both calls valid.

First call output:

```
Ruth  
Bob  
Guy  
Anita
```

Second call output:

```
Ruth  
Bob  
Guy  
Anita  
Mary
```

Parameter matching: `name1` = "Ruth"; remaining arguments go into rest array `names` (`["Bob", "Guy", "Anita"]` or with "Mary" added).

(iii)

```
function names(fname:string, lname?:string) { }  
names("Judith");  
names("Judith", "Bob");
```

Yes — both valid.

- `names("Judith")` — `fname` = "Judith", `lname` is **undefined** (optional). No output; function body empty.
 - `names("Judith", "Bob")` — `fname` = "Judith", `lname` = "Bob".
-

(d) (4 + 3 = 7 marks) — `printDimensions`

(d-i)

```
interface Dimensions {
  xPos: number;
  yPos: number;
}
function printDimensions(dims: Dimensions) {
  console.log(/* B */);
  console.log(/* C */);
}
/* A */
```

Part	Answer
------	--------

(i) A	<code>printDimensions({ xPos: 3, yPos: 4 });</code>
-------	---

(ii) B	<code>console.log("x = " + dims.xPos);</code> or <code>console.log(`x = \${dims.xPos}`);</code>
--------	---

(iii) C	<code>console.log("y = " + dims.yPos);</code> or <code>console.log(`y = \${dims.yPos}`);</code>
---------	---

Console output:

```
x = 3
y = 4
```

(d-ii)

```
interface Dimensions {
  xPos: number;
  yPos: number;
  zPos?: number;
}
function printDimensions(dims: Dimensions) {
  console.log(/* B — same as (i) */);
  console.log(/* C — same as (i) */);
  /* D */
}
/* A */
```

Part	Answer
------	--------

(iv) A	<code>printDimensions({ xPos: 3, yPos: 4 });</code> (same as d-i)
--------	---

(v) B	<code>console.log("x = " + dims.xPos);</code>
-------	---

(vi) C	<code>console.log("y = " + dims.yPos);</code>
--------	---

For D (optional `zPos` — typical exam expectation):

```
if (dims.zPos !== undefined) {
  console.log("z = " + dims.zPos);
}
```

If **A** passes only { xPos: 3, yPos: 4 }, output is still only x = 3 and y = 4 (no z line).

Question 3 — Angular (25 marks)

Assumes the angular.io framework.

(a) (4 marks) — Roles of files in src/app

File	Role
i) app.component.css	Component-specific styles — CSS rules that apply only to this component's template (scoped to AppComponent).
ii) app.component.html	Component template — HTML markup that defines the structure/view of the root app component (when not using inline template).
iii) app.component.ts	Component class — TypeScript logic: metadata (@Component), properties, methods, and behavior for the root component.
iv) styles.css	Global styles — CSS applied application-wide (not limited to one component).

(b) (4 + 2 = 6 marks)

```
import {Component} from '@angular/core';
@Component({
  selector: 'app-root',
  imports: [],
  template: `
    <h1>Default</h1>
  `,
  styleUrls: ['./app.component.css'],
})
export class AppComponent {
  title = 'default';
}
```

(i) Role of metadata

Metadata	Role
selector	CSS selector (app-root) used to insert this component into a parent template (e.g. <app-root></app-root> in index.html).

Metadata	Role
imports	Array of standalone dependencies (other components, directives, pipes) this component needs in its template.
template	Inline HTML defining the component's view (here, an <code><h1>Default</h1></code>).
styleUrls	Paths to external CSS files for this component's styles.

(ii) Last three lines (max 2 sentences)

The **export class AppComponent** declares the root component class that Angular instantiates. The property **title = 'default'** initializes a string field on the component instance that can be bound in the template (e.g. interpolation), though the shown template currently displays the literal text "Default" instead of `{{ title }}`.

(c) (2.5 marks) — New component **home**

Item	Answer
i) Directory	<code>src/app/home/</code> (or <code>src/app/home</code> under app)
ii) .ts file name	<code>home.component.ts</code>
iii) Class name	<code>HomeComponent</code>
iv) selector value	<code>'app-home'</code> (Angular convention: <code>app-</code> + component name)

(d) (1.5 + 2 + [2+1+6] structure — integration in **app.component.ts**)

(Paper asks to add/update `app.component.ts` — note: question text says `app.components.ts` in one place; standard file is `app.component.ts`.)

(i) Import statement

```
import { HomeComponent } from './home/home.component';
```

(ii) Modification to **imports** metadata

```
@Component({
  selector: 'app-root',
  imports: [HomeComponent],
  ...
})
```

(iii) Add home component to app view

In `template` (or `app.component.html`):

```
<app-home></app-home>
```

Or combined inline template example:

```
template: `
  <h1>Default</h1>
  <app-home></app-home>
`,
imports: [HomeComponent],
```

(e) (2 + 1 + 6 marks) — `HomeLocation` interface

File `homelocation.ts`:

```
export interface HomeLocation {
  id: number;
  street: string;
  town: string;
  region: string;
}
```

(i) First line in one sentence

`export interface HomeLocation` declares a **public TypeScript interface** named `HomeLocation` that can be imported by other files to type-check objects with `id`, `street`, `town`, and `region` properties.

(ii) Import in `app.component.ts`

```
import { HomeLocation } from './homelocation';
```

(Adjust path if file is in a subfolder, e.g. `./homelocation` or `./models/homelocation`.)

(iii) Property on `AppComponent`

```
homeLocation: HomeLocation = {
  id: 50,
  street: 'Street 20',
  town: 'Buea',
  region: 'SW Region'
};
```

(f) Template fragments — bindings (remaining marks)

Fragment (a)

```
<app-housing-location
  *ngFor="let housingLocation of housingLocationList"
  [housingLocation]="housingLocation">
</app-housing-location>
```

(i) Selector of embedded component:

app-housing-location (from custom element tag name; matches component `selector: 'app-housing-location'`).

(ii) What **housingLocation** refers to in `[housingLocation]="housingLocation"`:

The **current item** from the `*ngFor` loop — each element of the array `housingLocationList` as the loop iterates (loop variable `housingLocation`).

(iii) What is rendered (1–2 lines):

For **each** item in `housingLocationList`, Angular creates one **app-housing-location** child component and passes that item into its `housingLocation` input — so you get **one housing-location view per array element**.

Fragment (b)

```
<ul>
  <li *ngFor="let hero of heroes">
    <button type="button" (click)="selectHero(hero)">
      {{hero.name}}
    </button>
  </li>
</ul>
```

(iv) What is rendered:

An **unordered list** of **buttons**, one per hero; each button shows that hero's **name**, and clicking it calls `selectHero(hero)` with that hero object.

(v) Identify binding types

Binding	Example in code
Property binding	<code>[housingLocation]="housingLocation"</code>
Event binding	<code>(click)="selectHero(hero)"</code>

Binding	Example in code
Interpolation	<code>{{hero.name}}</code>

(Also structural directive **ngFor* drives repetition — not a binding type in the strict three-way list.)

(vi) Brief explanation of each

Binding	Explanation
Property binding	Sets a property/input on a child component or DOM element from a parent expression (one-way: component → view). <code>[housingLocation]="housingLocation"</code> passes data into the child.
Event binding	Listens for a DOM/component event and runs a handler in the component (view → component). <code>(click)="selectHero(hero)"</code> runs <code>selectHero</code> when the button is clicked.
Interpolation	Embeds a component property/expression into the template as text: <code>{{hero.name}}</code> displays the hero's name in the button label.

End of model answers — Questions 1–3, Second Semester Exam 26/06/2025.